Ajay Patel

.NET

10 Real Problems You Can Fix with C# Pattern Matching

Ajay Patel

❌ The classic null check issue

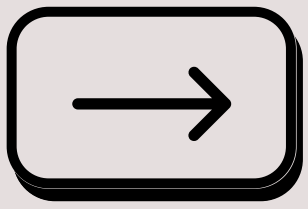
Problem: A null check passed, but type conversion still failed at runtime.

```
if (obj != null && obj is Customer)
{
    var customer = (Customer)obj;
}
```

✅ Type Pattern (C# 7.0)

Introduced in C# 7.0, type patterns allow safe type checking and casting in a single step.

```
if (obj is Customer customer)
{
    //No need for explicit type casting
}
```



Ajay Patel

❌ Complex switch-case logic cluttering the code

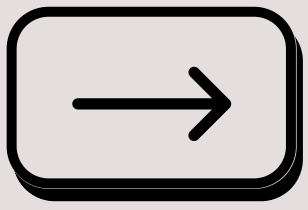
Problem: Too many case statements made maintenance difficult, introducing subtle branching bugs.

```
switch (fruit)
{
    case "Apple":
        Console.WriteLine("Red");
        break;
    case "Banana":
        Console.WriteLine("Yellow");
        break;
    case "Grapes":
        Console.WriteLine("Purple");
        break;
    default:
        Console.WriteLine("Unknown color");
        break;
}
```

✅ Switch Expression (C# 8.0)

Introduced in C# 8.0, switch expressions allow concise pattern matching.

```
var color = fruit switch
{
    "Apple" => "Red",
    "Banana" => "Yellow",
    "Grapes" => "Purple",
    _ => "Unknown color"
};
```



Ajay Patel

❌ Enum checks causing unnecessary boilerplate

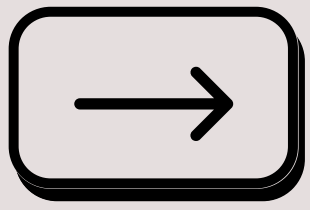
Problem: A missing enum check led to unintended execution in certain conditions.

```
if (status == Status.Active || status == Status.Pending)
{
    Console.WriteLine("Process allowed");
}
```

✅ Logical Patterns (C# 9)

Introduced in C# 9.0, logical patterns simplify multiple comparisons.

```
if (status is Status.Active or Status.Pending)
{
    Console.WriteLine("Process allowed");
}
```



Ajay Patel

5

❌ Nested if-else blocks making readability suffer

Problem: Redundant condition checks made logic difficult to follow and prone to edge-case failures.

```
if (obj is Employee)
{
    var emp = (Employee)obj;
    if (emp.Department == "HR")
    {
        Console.WriteLine("HR employee found");
    }
}
```

✅ Property Pattern (C# 8.0)

Introduced in C# 8.0, property patterns allow direct checks on object properties.

```
if (obj is Employee { Department: "HR" })
{
    Console.WriteLine("HR employee found");
}
```

Ajay Patel

❌ Multiple property conditions causing clutter

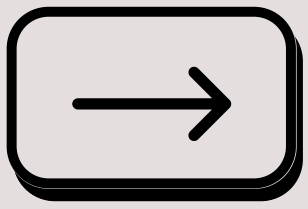
Problem: Checking multiple properties made the logic verbose.

```
if (customer != null && customer.Age > 18 && customer.Location == "US")
{
    Console.WriteLine("Eligible customer");
}
```

✅ Extended Property Pattern (C# 10)

Introduced in C# 10, this allows checking multiple properties directly in one pattern.

```
if (customer is { Age: > 18, Location: "US" })
{
    Console.WriteLine("Eligible customer");
}
```



Ajay Patel

❌ Overcomplicated string pattern matching

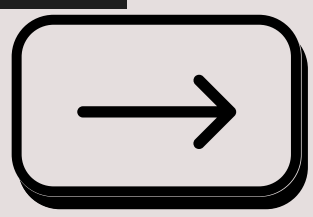
Problem: String checks required multiple conditions, making code harder to follow.

```
if (command == "Start" || command == "Run" || command == "Begin")
{
    Console.WriteLine("Executing action...");
}
else if (command == "Stop" || command == "End" || command == "Terminate")
{
    Console.WriteLine("Stopping action...");
}
else
{
    Console.WriteLine("Unknown command");
}
```

✅ Pattern-matching improvements (C# 9)

Introduced in C# 9, With pattern combinators, you can combine patterns via three new keywords (and, or, and not):

```
var commandResult = command switch
{
    "Start" or "Run" or "Begin" => "Executing action...",
    "Stop" or "End" or "Terminate" => "Stopping action...",
    _ => "Unknown command"
};
```



Ajay Patel

❌ Complex number range checks

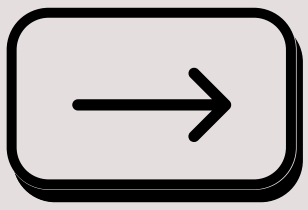
Problem: Checking for a valid numeric range required redundant comparisons.

```
if (score >= 50 && score <= 100)
{
    Console.WriteLine("Valid score range");
}
```

✅ Relational+Logical Patterns (C# 9)

Introduced in C# 9, this allows the <, >, <=, and >= operators to appear in patterns:

```
if (score is >= 50 and <= 100)
{
    Console.WriteLine("Valid score range");
}
```



Ajay Patel

❌ Verbose tuple checks

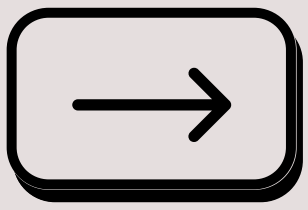
Problem: Checking values inside a tuple required separate conditions

```
if (coordinates.X == 10 && coordinates.Y == 20)
{
    Console.WriteLine("Match found");
}
```

✅ Tuple Pattern Matching (C# 8)

Introduced in C# 8. Tuple patterns allow cleaner multiple condition checks.

```
if (coordinates is (10, 20))
{
    Console.WriteLine("Match found");
}
```



Ajay Patel

❌ Boolean flag checks requiring extra logic

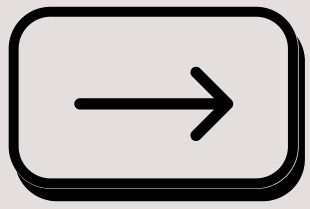
Problem: Boolean conditions required extra verification before performing an action

```
if (isActive && isVerified)
{
    Console.WriteLine("User has access.");
}
else if (!isActive || !isVerified)
{
    Console.WriteLine("Access denied.");
}
```

✅ Tuple Pattern Matching (C# 8)

Introduced in C# 8. Tuple patterns allow cleaner multiple condition checks.

```
var result = (isActive, isVerified) switch
{
    (true, true) => "User has access.",
    (false, _) or (_, false) => "Access denied."
};
```



Ajay Patel

❌ Checking for multiple values in an array required loops

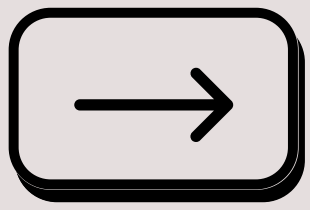
Problem: Finding if an array contains specific values was verbose.

```
int[] validValues = { 10, 20, 30 };
if (validValues.Contains(value))
{
    Console.WriteLine("Value is valid");
}
```

✅ List Pattern Matching (C# 11)

Introduced in C# 11. A list pattern matches a series of elements in square brackets

```
if (validValues is [_,>=20,30])
{
    Console.WriteLine("Value is valid");
}
```



Ajay Patel

.NET

Knowledge is contagious, let's spread it!

DO YOU LIKE THIS POST?

REPOST IT!

THANKS FOR READING